

Guía Pedagógica: Día 4 - Home: El "Hero" de la Pantalla

Hoy construiremos el componente más impactante de nuestra página de inicio: el Hero Section. Es esa gran imagen de fondo con información de la película destacada que atrapa al usuario nada más entrar en aplicaciones como Netflix o HBO.

Contexto Pedagógico: Smart Components vs. Dumb Components

En Angular profesional, es una excelente práctica dividir nuestros componentes en dos grandes grupos para mantener el código limpio y mantenible:

1. **Smart Components (Contenedores):** Como nuestro Home. Ellos conocen el "estado" de la aplicación, inyectan servicios y se encargan de pedir los datos a la API.
2. **Presentational Components (Dumb Components):** Como el Hero que construiremos hoy. Ellos no saben de dónde vienen los datos, solo reciben un objeto (la película) y se encargan de mostrarlo de forma hermosa. Son altamente reutilizables.

Objetivo:

1. **Generar el Componente Hero:** Nuestra pieza de presentación (Dumb Component).
2. **Comunicación @Input:** Aprenderemos cómo el padre (Home) le "pasa" la información al hijo (Hero).
3. **Diseño Visual de Impacto:** Construiremos el HTML y el CSS (usando degradados y backgrounds dinámicos) para que luzca espectacular.
4. **Integración en Home:** Conectaremos la API (Smart Component) con nuestro nuevo Hero.

Paso 1: Generando y Configurando el Componente Hero

Vamos a crear el lugar donde vivirá la película destacada.

1. Generar el componente

Abre tu terminal y ejecuta el siguiente comando para crear el componente dentro de la funcionalidad (feature) de home:

```
ng g c features/home/components/hero
```

2. La Lógica (TypeScript) y el poder del @Input

Abre **src/app/features/home/components/hero/hero.ts**. Dado que el Hero es un componente de presentación, necesita que alguien le pase la información. Para esto usamos el decorador @Input().

```
import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Movie } from '../../../core/models/movie.model';

@Component({
  selector: 'app-hero',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './hero.html',
  styleUrls: ['./hero.css']
})
export class Hero {
  // ¡Recibimos la película desde el padre (Home)!
  // 'required: true' obliga a que no se pueda usar <app-hero> sin
  // pasarle una película
  @Input({ required: true }) movie!: Movie;

  // Getter profesional para construir la URL de la imagen de fondo de
  // alta calidad
  get backdropUrl() {
    return this.movie.backdrop_path
      ? `https://image.tmdb.org/t/p/original${this.movie.backdrop_path}`
      : ''; // URL de respaldo si es necesario
  }
}
```

Paso 2: La Vista y el Diseño de Impacto (HTML y CSS)

El secreto de un buen Hero es asegurar que el texto sea legible sin importar qué tan clara o compleja sea la imagen de fondo de la película.

1. La Estructura (HTML)

Abre hero.html. Usaremos **Property Binding** [style.background-image] para inyectar dinámicamente la URL de la película directamente en el CSS del componente.

Además, integraremos las clases base que construimos en el Día 3 (.text-gradient, .btn-premium).

```
<!-- Inyectamos el fondo dinámicamente usando la variable de TypeScript -->
<section class="hero" [style.background-image]='url(' + backdropUrl +
  ')''>

  <!-- Este div crea un degradado oscuro para que el texto sea legible -->
  <div class="hero-overlay"></div>

  <div class="hero-content">
    <div class="hero-info">
      <span class="badge">Película Destacada</span>

      <!-- Usamos nuestra clase reutilizable del Sistema de Diseño (Día 3) -->
      <h1 class="hero-title text-gradient">{{ movie.title }}</h1>

      <p class="hero-overview">{{ movie.overview }}</p>

      <div class="hero-actions">
        <!-- Reutilizamos nuestros botones premium -->
        <button class="btn-premium">
          <span>Reproducir</span>
        </button>
        <button class="btn-outline">
          <span>Más información</span>
        </button>
      </div>
    </div>
  </div>
</section>
```

2. Los Estilos (CSS)

Abre **hero.css**. Aquí aplicaremos un concepto clave: **linear-gradient**. Este degradado irá de negro (abajo, para fundirse con la web) a transparente (arriba, para ver la imagen).

```

/* Contenedor principal del Hero */
.hero {
  position: relative;
  width: 100%;
  height: 85vh; /* Ocupa el 85% del alto de la pantalla */
  background-size: cover;
  background-position: center top;
  display: flex;
  align-items: flex-end; /* Alineamos el contenido hacia abajo */
}

/* EL secreto de la legibilidad: EL Overlay */
.hero-overlay {
  position: absolute;
  inset: 0; /* shorthand para top:0, right:0, bottom:0, left:0 */
  /* Degradado que va de negro sólido en la base a transparente arriba */
  background: linear-gradient(
    to top,
    var(--bg-color) 0%,
    rgba(10, 10, 10, 0.8) 30%,
    transparent 100%
  );
  z-index: 1;
}

/* EL contenido (texto y botones) */
.hero-content {
  position: relative;
  z-index: 2; /* Debe estar por encima del overlay */
  padding: var(--space-xl) var(--space-md);
  width: 100%;
  max-width: 1200px;
  margin: 0 auto;
}

.hero-info {
  max-width: 600px; /* Evitamos que el texto cruce toda la pantalla */
}

.badge {
  display: inline-block;
  background-color: rgba(255, 255, 255, 0.1);
  backdrop-filter: blur(4px);
  padding: 0.25rem 0.75rem;
  border-radius: 20px;
  font-size: 0.8rem;
  text-transform: uppercase;
  letter-spacing: 1px;
  margin-bottom: 1rem;
  border: 1px solid rgba(255, 255, 255, 0.2);
}

```

```

.hero-title {
  font-size: clamp(2.5rem, 5vw, 4.5rem);
  line-height: 1.1;
  margin-bottom: 1rem;
}

.hero-overview {
  font-size: 1.1rem;
  color: var(--text-muted);
  margin-bottom: 2rem;
  line-height: 1.6;
  /* Limita el texto a 3 líneas y pone '...' si es más largo */
  display: -webkit-box;
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.hero-actions {
  display: flex;
  gap: 1rem;
}

```

Paso 3: La Integración Final en el Componente Padre (Home)

Ya tenemos un Hero hermoso, pero está vacío. Nuestro componente Home (el Smart Component) es quien consumirá el MovieService y le entregará los datos al componente de presentación.

1. Actualizando la Lógica de Home

Abre **src/app/features/home/home.ts** y asegúrate de inyectar el servicio y guardar la primera película de la lista de tendencias.

```

import { Component, inject, OnInit, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { MovieService } from '../../core/services/movie.service';
import { Hero } from '../components/hero/hero'; // ¡Importamos el Hero!
import { Movie } from '../../core/models/movie.model';

@Component({
  selector: 'app-home',
  standalone: true,
  imports: [CommonModule, Hero], // Lo añadimos aquí
  templateUrl: './home.html',
  styleUrls: ['./home.css'],
})
export class Home implements OnInit {
  private movieService = inject(MovieService);

  // Usamos una Signal para guardar la película destacada
  featuredMovie = signal<Movie | null>(null);

  ngOnInit(): void {
    this.movieService.getTrendingMovies().subscribe({
      next: (data) => {
        if (data.results.length > 0) {
          // Tomamos la posición [0] del array para ser el Hero
          this.featuredMovie.set(data.results[0]);
        }
      }
    });
  }
}

```

2. Actualizando la Vista de Home

Abre **home.html**. Aquí es donde ocurre la comunicación Padre -> Hijo usando Property Binding ([movie]="..").

```
<!-- Verificamos si ya llegó la información de la API usando el control flow -->
@if (featuredMovie(); as movie) {
  <!-- Padre (Home) pasándole la variable 'movie' al Hijo (Hero) -->
  <app-hero [movie]="movie"></app-hero>
} @else {
  <!-- Mientras llega la información, mostramos un estado de carga (Skeleton) -->
  <div class="loading-hero">Cargando...</div>
}
```

Prueba de Aprendizaje (Checklist para el Aprendiz)

1. **¿Cuál es la diferencia entre un componente Smart y uno Dumb (de presentación)?** (Respuesta: Los Smart consultan datos de servicios/API y manejan el estado general. Los Dumb solo reciben datos por @Input y se enfocan puramente en cómo se ve la interfaz, haciéndolos súper reutilizables).
2. **¿Por qué usamos @Input({ required: true }) en el Hero?** (Respuesta: Para que el compilador de Angular nos obligue a pasarle un objeto Movie cada vez que usemos la etiqueta <app-hero>. Esto previene que el componente se rompa en producción por intentar leer datos "undefined").
3. **¿Por qué es fundamental el linear-gradient en el CSS de un componente tipo Hero?** (Respuesta: Porque la imagen de fondo viene de una API y es impredecible (puede ser muy clara o muy oscura). El degradado oscuro en la base asegura un contraste adecuado para que los textos blancos sean 100% legibles siempre).

PELÍCULA DESTACADA

Intercambiados

Una pequeña criatura del bosque intercambia el cuerpo con un majestuoso pájaro del valle, lo que los obliga a colaborar para sobrevivir a la aventura más salvaje de sus vidas.

[Reproducir ahora](#)[Más información](#)

Control de Versiones

Finaliza tu día con:

```
git add .  
git commit -m "feat: implementar componente Hero con datos dinámicos de la API"  
git push origin main
```